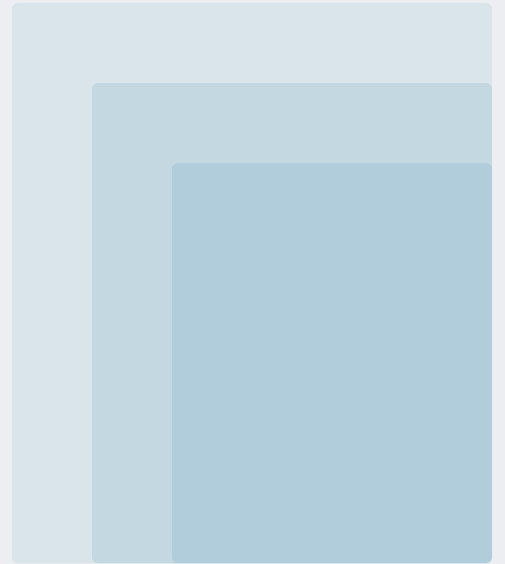




Final Project Guidelines

Data Science Programming Languages (Lab)

Your journey from data to insights starts here



Table of Contents

Table of Contents	2
Welcome to Your Project	4
What You Will Learn	4
Project Roadmap	4
Task 1: Choose Your Dataset	5
Objective.....	5
Requirements Checklist	5
Getting Started	5
Task 2: Kaggle API & Data Loading	6
Objective.....	6
Step-by-Step Guide.....	6
What to Include in Your Notebook	6
Task 3: Python Core Skills	6
Objective.....	6
Required Demonstrations.....	7
Task 4: Object-Oriented Programming	7
Objective.....	7
Requirements	7
Example Structure	7
Task 5: Exception Handling	8
Objective.....	8
Requirements	8
Common Exception Ideas by Domain.....	8
Task 6: NumPy Operations.....	8
Objective.....	8
Required Operations.....	8
Task 7: Pandas Data Cleaning.....	9

Objective.....	9
Cleaning Checklist.....	9
Task 8: Data Analysis	9
Objective.....	9
What Makes a Good Analysis Question?.....	9
Analysis Techniques to Use.....	10
Example Questions	10
Task 9: Excel Export	10
Objective.....	10
Excel File Structure	10
Formatting Tips.....	10
Submission Checklist.....	11
Tips for Success.....	11
Code Quality	11
Going Above and Beyond	11

Welcome to Your Project

Congratulations on reaching the final project! This is your opportunity to bring together everything you have learned throughout this course and apply it to a real-world dataset of your choice. Think of this project as your data science portfolio piece — a chance to showcase your skills and creativity.

You will work through nine structured tasks, each designed to build upon the previous one. By the end, you will have a complete data analysis pipeline: from downloading raw data to producing a polished Excel report.

Tip: Start Early: Give yourself plenty of time to find an interesting dataset. The best projects come from genuine curiosity about the data!

What You Will Learn

- How to source and load real datasets using professional APIs
- Core Python programming: lists, dictionaries, functions, and comprehensions
- Object-oriented design for modeling real-world data entities
- Robust error handling with custom exceptions
- Numerical computing with NumPy and data manipulation with Pandas
- Data cleaning techniques for real-world messy data
- Exploratory data analysis and meaningful questioning
- Professional reporting with formatted Excel exports

Project Roadmap

Here is a visual overview of your journey through this project. Each task builds on the previous ones, leading you from raw data to insights:

Task	Topic	Key Skills
1	Dataset Selection	Kaggle, CSV, data exploration
2	Kaggle API & Loading	API config, pandas, data inspection
3	Python Core Skills	Lists, dicts, map, filter, lambda
4	Object-Oriented Programming	Classes, inheritance, polymorphism
5	Exception Handling	Custom exceptions, error handling

Task	Topic	Key Skills
6	NumPy Operations	Arrays, vectorization, statistics
7	Data Cleaning	Missing values, types, duplicates
8	Data Analysis	Filtering, grouping, insights
9	Excel Export	Multi-sheet reporting

Tip: Pro Tip: Keep your code organized with clear section headers and comments. This makes debugging easier and your notebook more professional.

Task 1: Choose Your Dataset

Objective

Select a public CSV dataset from Kaggle that meets all requirements below. The dataset you choose will be the foundation for your entire project, so pick something that genuinely interests you!

Requirements Checklist

Requirement	Details
Format	CSV (Comma-Separated Values)
Minimum Rows	At least 500 rows of data
Minimum Columns	At least 8 columns
Data Types	Must include both numerical and categorical columns
Download Method	Via Kaggle API (not manual download)

Getting Started

0. **Browse Kaggle Datasets:** Visit [kaggle.com/datasets](https://www.kaggle.com/datasets) and explore categories like Health, Finance, Sports, Environment, or Technology.
1. **Check the Shape:** Before downloading, verify the dataset has at least 500 rows and 8 columns by reading the dataset description.
2. **Verify Data Types:** Preview the dataset to confirm it contains both numerical (integers, floats) and categorical (text, categories) columns.
3. **Download via API:** You will use the Kaggle API (not manual download) — this is covered in Task 2.

Tip: Dataset Ideas: Housing prices, COVID-19 statistics, movie ratings, customer churn, weather data, or sports statistics. Choose what excites you!

Task 2: Kaggle API & Data Loading

Objective

Configure the Kaggle API, download your chosen dataset programmatically, load it into a pandas DataFrame, and perform an initial exploration of its structure.

Step-by-Step Guide

4. **Get Your API Key:** Go to kaggle.com/account, scroll to "API" section, and click "Create New API Token." This downloads a `kaggle.json` file.
5. **Configure Credentials:** Place `kaggle.json` in `~/.kaggle/` (Linux/Mac) or `C:\Users\<YourName>\.kaggle\` (Windows). Set permissions if needed.
6. **Install Kaggle Library:** Run: `pip install kaggle` in your terminal or notebook cell.
7. **Download Dataset:** Use the Kaggle API command or Python library to download your dataset directly into your notebook environment.
8. **Extract & Load:** Extract the ZIP file and load the CSV using `pandas.read_csv()`.
9. **Initial Exploration:** Display: dataset shape, first 5 rows, column names, and data types.

What to Include in Your Notebook

- Code to download and extract the dataset using Kaggle API
- pandas code loading the CSV into a DataFrame
- Output showing: `df.shape`, `df.head()`, `df.columns`, `df.dtypes`
- A brief explanation of what one row in your dataset represents (1-2 sentences)

Task 3: Python Core Skills

Objective

Demonstrate your mastery of Python fundamentals using only the first 20 rows of your dataset. This section is your chance to show you can manipulate data with pure Python — no pandas shortcuts allowed here!

Required Demonstrations

- Lists: Create and manipulate lists from your data (e.g., extract column values)
- Dictionaries: Build dictionaries from your data rows
- List Comprehensions: Transform or filter data using [x for x in data if condition]
- map(): Apply a function to every element in a sequence
- filter(): Select elements matching a condition
- Lambda Expressions: Write concise anonymous functions
- Sorting: Sort your data by custom criteria
- Custom Function: Write at least one reusable function with docstring

Tip: Focus on the first 20 rows only. This keeps computation fast and your notebook readable. Show clear before/after outputs for each operation.

Task 4: Object-Oriented Programming

Objective

Design a class hierarchy that models entities from your dataset. This demonstrates your ability to translate real-world data into clean, reusable code structures.

Requirements

- One Parent Class: Define a base class with shared attributes and methods relevant to your dataset
- Two Child Classes: Create two subclasses that inherit from the parent and add specialized behavior
- Encapsulation: Use private attributes (prefix with `_`) and provide getter/setter methods
- Inheritance: Child classes must properly call the parent constructor
- Polymorphism: Implement at least one method that behaves differently in each child class

Example Structure

If your dataset is about movies: a parent `Media` class with child classes `Movie` and `Series`. The shared method `display_info()` could show different details for each type.

Tip: Create at least 2-3 objects from your actual dataset rows to demonstrate your classes working with real data.

Task 5: Exception Handling

Objective

Create a custom exception class tailored to your dataset, then demonstrate its use in a realistic scenario. This shows you understand how to build robust, production-quality code.

Requirements

- Create a custom exception class (e.g., `InvalidDataError`, `InvalidAgeError`, `MissingFieldError`)
- The exception should be meaningful to your specific dataset domain
- Demonstrate raising and catching your exception with a practical example

Common Exception Ideas by Domain

- Health data: `InvalidRangeError` for impossible vital signs
- Financial data: `NegativeValueError` for prices or balances
- Date fields: `InvalidDateError` for future dates in historical data
- Categorical data: `InvalidCategoryError` for unexpected values

Task 6: NumPy Operations

Objective

Convert numerical columns from your dataset into NumPy arrays and perform key numerical operations. This task demonstrates your ability to work with efficient numerical computing in Python.

Required Operations

- Convert at least two numerical columns to NumPy arrays using `df['column'].values`
- Vectorized Calculation: Perform at least one mathematical operation across entire arrays (e.g., addition, multiplication, or custom formula)
- Boolean Filtering: Select elements meeting a condition (e.g., values greater than a threshold)
- Statistical Summary: Calculate and display mean, minimum, maximum, and standard deviation for your arrays

Tip: Choose columns that relate to each other for more interesting analysis. For example, if analyzing housing data, compute price per square foot from two separate columns.

Task 7: Pandas Data Cleaning

Objective

Clean and prepare your dataset for analysis. Real-world data is always messy — this task shows you can transform raw data into a structured, analysis-ready format.

Cleaning Checklist

Step	What to Do
Column Names	Rename to snake_case or descriptive names (no spaces)
Missing Values	Identify and handle: drop, fill, or impute
Duplicates	Detect and remove duplicate rows
Data Types	Convert columns to appropriate types (int, float, datetime, category)
Consistency	Standardize text values (e.g., lowercase, trim whitespace)
Outliers	Note any extreme values (optional: handle them)

Tip: Document every cleaning step with comments explaining WHY you made each change. This shows your analytical thinking and makes your work reproducible.

Task 8: Data Analysis

Objective

Formulate and answer three meaningful analysis questions about your dataset. This is the heart of your project — where you transform clean data into genuine insights.

What Makes a Good Analysis Question?

- **Specific:** Not "analyze the data" but "What is the average price by neighborhood?"
- **Answerable:** Your dataset must contain the needed information
- **Insightful:** The answer should reveal something interesting or actionable

Analysis Techniques to Use

- Filtering: `df[df['column'] > value]` to focus on subsets
- Grouping: `df.groupby('category')` to aggregate by categories
- Aggregation: `.mean()`, `.sum()`, `.count()` for summary statistics
- Sorting: `df.sort_values()` to rank results

Example Questions

- Which category has the highest average value?
- What is the trend over time (if date column exists)?
- Are there correlations between two numerical variables?

Tip: After each analysis, write 2-3 sentences interpreting your results. Raw numbers are not enough — explain what they mean in context!

Task 9: Excel Export

Objective

Export your cleaned dataset, analysis results, and summary into a single, well-formatted Excel file. This demonstrates your ability to create professional deliverables that stakeholders can easily read and use.

Excel File Structure

Sheet	Contents
Cleaned Data	Your fully cleaned dataset, ready for analysis
Analysis	Pivot tables or grouped results from Task 8
Summary	Key findings, statistics, and conclusions

Formatting Tips

- Use meaningful sheet names (not Sheet1, Sheet2)
- Add bold headers and column widths that fit the content
- Consider adding a title row above your data on the summary sheet

Submission Checklist

Before submitting, verify that you have completed all of the following:

Deliverable	Notes
[] Jupyter Notebook (.ipynb)	All 9 tasks completed with outputs visible
[] Excel Report (.xlsx)	Named Project_Report.xlsx, 3 sheets
[] Code Runs	Restart kernel and run all cells successfully
[] No CSV Submitted	Dataset downloads via API in the notebook
[] Comments & Markdown	Explanations between code sections

Tip: Do NOT submit the downloaded CSV file. Your notebook should download it automatically when run. Test this by restarting your kernel and running all cells!

Tips for Success

Code Quality

- Use clear, descriptive variable names
- Add markdown cells between code sections to explain your approach
- Include comments for complex logic
- Restart and run all cells before submitting to ensure everything works

Going Above and Beyond

- Create data visualizations (matplotlib/seaborn) — not required but impressive!
- Add a conclusions section summarizing your key findings
- Include suggestions for future analysis or data collection

Note: Students may work individually or in a team of up to three members; a maximum of three students per team.

You have got this! Every data scientist started with their first project.

Make it count!

•

Good Luck!

We are here to help you succeed.

